

Lesson 9, Part 2 Activity: Estimate One Slice of Your Project

CS 5388 / CS 4390 Software Project Management | Team activity, complete together

Team: Team 9 **Project:** GPI: Gracias Por Invitar

What this activity is for

Your Sprint 2 business case has to answer a question every sponsor asks: how much work would this take, and what would it cost? You are not going to build the system in this course. You are doing what a project manager does before a project is approved: estimating the work so a decision can be made.

Estimating a whole project at once hides mistakes. So, you will start small. Pick one or two major parts of your project, called a slice, and estimate only that slice. A small slice lets you count everything, check every number, and see clearly where your assumptions matter.

You will estimate the same slice twice, using two methods that start from different information. Then you will compare the two answers, explain why they differ, and present a range with your assumptions written down. That range becomes the Estimation Appendix on your Sprint 2 portal page.

What to bring

1. Your Release 1 capability list (Step 1 of the Part 1 starter).
2. Your backlog from the Part 1 activity: user stories, MoSCoW categories, and planning poker point sizes.
3. Your similar-products research from Part 1 (small and big products, with timelines and team sizes).
4. The Part 2 slides, especially Slides 13 to 17 (sizing), 19 to 24 (team size), 25 to 27 (story points), and 31 to 33 (reconciling).

How the activity fits together

Step	What you do	What you end up with
1	Pick your slice	One or two major parts of your project, with their stories, screens, and data listed
2	Choose two estimation methods	Two methods that use different starting information
3	Estimate the slice with each method	Two effort estimates, both in team hours
4	Check the estimates for team size	Confidence that coordination costs are included
5	Compare the two estimates	A written explanation of why they differ
6	Present the result	A dated range, in hours, dollars, and weeks, ready for the business case

Key terms

Term	What it means in this activity
Slice	One or two major parts of your project that a user would recognize, such as “buy a permit.” You estimate only this.
Size	How much software the slice contains, measured in function points, use-case points, or story points.
Effort	Total team hours of work. If 5 people each work 10 hours, that is 50 hours of effort.
Productivity rate	Hours of effort per unit of size, such as 8 hours per function point. The rate drives the whole estimate.
Assumption	Any number you chose rather than measured. Label it, so a reader knows it could change.
Velocity	Story points a team completes per sprint. You have no sprint history yet, so yours is an assumption.
Loaded rate	The full hourly cost of one person: salary plus benefits, overhead, and tools.
Range	A low and a high estimate. Honest estimates are ranges, not single numbers.
O, M, P	Optimistic, most likely, and pessimistic estimates, used in the PERT formula $(O + 4M + P) \div 6$.

About the examples

Every italic example uses the same slice from the fictional Campus Parking System: “Buy a semester permit.” Following one slice through every method shows how the pieces connect. Your own numbers come from your project.

Step 1. Pick your slice

A good slice is a part of your project that a user would recognize, and that is useful on its own. It usually covers 3 to 8 user stories from your backlog.

How to choose

1. Start with a Must-have capability that is central to your project. If it did not exist, the project would not make sense.
2. If you pick a second part, make it different in kind, such as one part users interact with and one part that produces reports or connects to another system.
3. Do not pick the easiest part. The slice should include some real complexity, such as payments, another system, or stored data.
4. Write down what is out of the slice, so your estimate has a clear boundary.

Question	Our slice
Slice name (one or two major parts)	User/Event goer features
Why we chose it	Event discovery is the central user experience. This coherent slice covers the calendar, date views, category preferences, saved events and search; organizer and admin flows are separate.
Screens or forms where users enter data	• Log in / User Registration • Account Settings Page • Category “interest” selection screen • Save Event Button
Reports, results, or lookups users see	• Outputs: Unified calendar of El Paso and Juárez events merged from different sources; feed ranked by the user's chosen categories. Personalization in this slice filters or orders events by user-selected categories; machine-learning and predictive recommendations are not included • Lookups: events listed by date; event details page (time, place, details); search results from the search bar; saved events list; my account and chosen categories
Data the slice stores	• Users • User's preferences • Events
Other systems it connects to	• External event sources the unified calendar pulls events from • Source of events: this slice consumes a common GPI events catalogue. For Release 1 the Project Charter and the Business Strategy have organizers posting events, with the team curating at first; this estimate also assumes one outside source read by the unified calendar, counted as one external interface file. Organizer-management screens are excluded, and any other ingestion is excluded. How many outside sources Release 1 reads, if any, is not yet decided (see Step 4 and Step 6).
Out of the slice (not included)	• Organizers uploading or editing events • Paid featured listings for organizers • Admin review and approval of submitted events • Ticketing and payment processing (out of scope in the charter) • Email or push notifications about new events • Social features, like a friends list or seeing who else is going

Stories in the slice

User story	MoSCoW	Points
As an event goer, I can see a unified calendar of events across El Paso and Juárez.	Must	8
As an event goer, I can browse upcoming events listed by date so I know what is on this weekend.	Must	5
As an event goer, I can open an event and see its time, place and details.	Must	3
As an event goer, I can create an account and subscribe to the event categories I care about.	Must	8
As an event goer, I can see a feed ranked to the categories I chose.	Should	5
As an event goer, I can save an event so I can come back to it later.	Could	3
As an event goer, I can search for an event by name or keyword.	Should	5
Total points		37

Points: the seven values were drafted for this activity rather than sized in a planning poker session by the whole team, and are to be confirmed by the team. Search was added to the story table in this revision; it was among the slice's screens before but had no story.

Step 2. Choose two methods

Each method starts from different information. Choose two that do not share the same starting point, so that if they agree, the agreement means something (Part 2, Slide 31).

Method	It starts from	A good choice if	Watch out for
Analogous	Similar products and how long they took	You found at least one product close to yours	Copying a big company's timeline without adjusting
Function points	Counts of what users enter, see, and store	Your Step 1 table lists screens, results, and data	A productivity rate with no source
Use-case points	The actors and use cases in your slice	You can list who uses the slice and for what	Default adjustment factors used without saying so
Story points to cost	Your planning poker points and a velocity	Every story in the slice has points	Velocity is a guess until you run sprints

Suggested pairs

1. Function points plus analogous: one starts from your own design, the other from the market. The examples below use this pair.
2. Story points plus analogous: one starts from your backlog, the other from the market.
3. Function points plus story points: one starts from system features, the other from team judgment of story size.

Our two methods: 1. Analogous 2. Function Points

Why these two use different starting information: Analogous method begins at the market research level where we identify facts such as failures and successes of a product. We can prove through research and facts about how an organization might have shipped a similar product and succeeded or failed. Function points, on the other hand, start from one's own design indicating count of inputs, results, lookups and data in our slices. Neither method uses the other's numbers, so if they agree, the agreement means something.

Step 3. Estimate the slice

Complete only the two sections (3A to 3D) that match your methods. Whatever method you use, finish with effort in team hours for your slice, so the two estimates can be compared in Step 5. Where you must choose a number, write it down and label it an assumption.

3A. Function points (Part 2, Slides 13 to 15)

What it measures: the functionality users get from the slice, counted in five categories. Your Step 1 table already lists most of what you need.

Category	What to count	Example from the permit slice
External Inputs (EI)	Forms where data enters the system	Register vehicle, purchase permit, update vehicle
External Outputs (EO)	Results the system calculates and sends	Permit confirmation and receipt
External Inquiries (EQ)	Simple lookups of stored data	Look up my permit, list lots with permits available
Internal Logical Files (ILF)	Groups of data the slice stores	Permit records, vehicle records
External Interface Files (EIF)	Data read from another system	Student registry, payment service

How to do it

1. Use your Step 1 table. Count each distinct input, output, lookup, and data group in the slice once.
2. Multiply each count by its weight and add the results. That total is your slice size in function points.
3. Choose a productivity rate in hours per function point. You have no history, so label it an assumption and test a low and a high value (Slide 15 shows 6 to 12 hours).
4. Multiply size by each rate to get a low and high effort in hours.

Category	Count	Weight	Points
External Inputs (EI)	4	4	16
External Outputs (EO)	2	5	10
External Inquiries (EQ)	5	4	20
Internal Logical Files (ILF)	3	10	30
External Interface Files (EIF)	1	7	7
Total function points			83

Rate: low 6 and high 12 hours per FP

Assumption or source: Used the 6 to 12 hour range from Part2, Slide 15 since we have no productivity history.

Effort: **83 FP × 6 h = 498 hours (low)**

83 FP × 12 h = 996 hours (high)

Our counts, itemized. External inputs (4): user registration, account settings update, category interest selection, and save event. Log in is listed as a screen in Step 1 but is not counted as an input, because it stores no business data of its own. External outputs (2): the unified calendar, which merges events from outside sources and resolves the straightforward duplicates, and the category-ranked feed, which orders events against stored preferences. Both derive data rather than return it as stored, which is what separates an output from an inquiry on Slide 13. External inquiries (5): events listed by date, the event detail page, search results, the saved events list, and the account and chosen categories view. Internal logical files (3): users, user preferences, and events. External interface file (1): the outside event source the unified calendar reads and

does not maintain, counted as one logical source; Step 6 records what changes if Release 1 reads none or several. Every item appears once and none is counted in two categories.

Example: 3 inputs (12), 1 output (5), 2 inquiries (8), 2 data groups (20), and 2 external systems (14) give 59 function points. At 6 to 12 hours per function point (assumed), effort is 354 to 708 hours; at 8 hours it is 472.

3B. Use-case points (Part 2, Slides 16 and 17) (Not applicable)

Not used since our two methods are analogous and function points.

3C. Story points to cost (Part 2, Slides 25 to 27)

What it measures: effort based on your team's own sizing of the stories in the slice. Story points are relative, so you convert them using a velocity (points per sprint). Because you have not run any sprints, velocity is an assumption. Use a low, middle, and high value.

How to do it

1. Use the total points from your Step 1 story table.
2. Choose a low, middle, and high velocity in points per two-week sprint. Label them assumptions.
3. Divide the slice points by each velocity. Keep the decimals; a small slice may take less than one sprint.
4. Multiply by team hours per sprint (team size $\times$ hours per person per sprint) to get effort in hours.

Input	Value	Note
Slice points	37	From your Step 1 story table
Team size	5	Five or six at most
Hours per person per sprint	20	State your assumption
Velocity: low / middle / high	4 / 6 / 8	Assumptions until we have sprint data
Sprints needed (high to low velocity)	4.6 / 6.2 / 9.3	Slice points $\div$ velocity
Effort in hours (low to high)	463 to 925 hours	Sprints $\times$ team size $\times$ hours per person per sprint

Our assumptions: Twenty hours per person per sprint is a two-week sprint at 10 productive hours a week each, alongside coursework. The velocities of 4, 6 and 8 points per sprint have nothing behind them: Slide 26 says real velocity needs at least three completed sprints by the same team, and we have run none. Effort is sprints $\times$ 5 people $\times$ 20 hours, so 100 team hours a sprint, giving 463 hours at the high velocity, 617 at the middle and 925 at the low. Low effort corresponds to the highest velocity.

Example: 21 points at 30, 20, or 15 points per sprint takes 0.7, 1.05, or 1.4 sprints. With 5 people working 80 hours each per sprint (400 team hours), effort is 280 to 560 hours.

3D. Analogous (Part 2, Slide 10)

What it measures: effort based on how long a similar real product took. You turn the product's timeline into person-months, scale it down to the size of your slice, then convert it to hours.

How to do it

1. Pick the similar product closest to your project from your Part 1 research, and note its team size and time to first release.
2. Multiply team size by months to get person-months. This is the effort the comparable product took.
3. Estimate what fraction of their first release your slice matches. If your slice is about a tenth of what they shipped, multiply by 0.1. Give a low and high fraction and explain them.
4. Adjust for team size. A much larger team spent more of its time coordinating, so its effort per feature is higher than yours will be.
5. Convert to hours. State the hours per person-month you assume (for example, 160).

Question	Answer
Comparable product, team size, and time to first release	Upcoming.org. One person (Andy Baio), about 9 months to first public release, 19 September 2003 (waxy.org launch post).
Person-months (team size $\times$ months)	$1 \times 9 = 9$ person-months. Baio built it nights and mornings around a full-time job, so we assume 70 hours per person-month rather than 160 (assumption). That puts their first release at roughly 630 hours.
Fraction of their release our slice matches (low to high), and why	50 to 70 percent. Their first release was browse-by-metro listings in fixed categories, user-added events, attendee lists, a friends list and RSS. Our slice takes the browsing and listing side: their metro browse against our calendar and date list, their event page against ours, and their events-you-are-going-to list against our saved events. We add account preferences and a category-ranked feed, which their first release did not have. Later updates added an API, tagging, email and SMS (the sources we listed do not date them), and we found no source showing it ever carried a ranked feed, so we do not claim one. We leave out their event entry, attendee lists and friends list, which sit outside our boundary. Their first release also had no import from outside sources: events were added by users. The outside-source import in our slice therefore has no counterpart in the comparable, so this fraction covers the browsing side only, and the analogous figure is likely low for as long as the import stays in the slice.
Team size adjustment and why	Uplift of 25 to 35 percent. Upcoming was one person with no coordination cost at all. We are five, with 10 communication paths, so our effort per feature is higher than his. We did not scale from Fever's four founders, whose first release took about two years by our Part 1 worksheet (a figure the listed sources do not confirm), because that was a company rather than a side project and its effort already carries its own coordination.

Question	Answer
Adjusted effort in hours (low to high)	630×0.50 to $0.70 = 315$ to 441 hours. With the 1.25 to 1.35 uplift, 394 to 595 hours. Middle, at 60 percent and 1.30 , is 491 hours.
Confidence (low, medium, high) and why	Medium. The comparable is the right shape, and the 19 September 2003 first release is grounded in Baio's own launch post on waxy.org. The January 2003 development start is secondhand, from one attendee's notes on a talk he gave in 2008, and we found nothing written by him giving that date. Beyond the dates, the 70 hours per person-month, the 50 to 70 percent fraction and the 25 to 35 percent uplift are all our modeling choices rather than recorded figures. The fraction moves the answer by about 40 percent across its range and the uplift by about 8 percent; the hours per person-month move it without limit: at 100 hours per person-month the range would be 563 to 851 hours, and at a full-time 160, 900 to 1,361.

Example: ParkEasy (fictional) took 4 people about 9 months, or 36 person-months, to launch a map of lots and pay-by-phone in one city. Buying a permit is roughly 8 to 12 percent of that release. 36×0.08 to $0.12 = 2.9$ to 4.3 person-months, or about 460 to 690 hours at 160 hours per person-month (about 576 hours at 10 percent).

Before you move on: put both estimates in hours
Step 5 compares your two estimates directly, so both must be in team hours for the same slice. If an estimate came out in months, sprints, or dollars, convert it now and show the conversion.

Step 4. Check your estimates for team size (Part 2, Slides 19 to 24)

Larger teams spend more of their time coordinating: meetings, handoffs, and fitting work back together. Estimators tend to underweight this cost, so check each estimate against the list below. Anything marked No needs a fix or a written explanation.

Check	Yes / No	Why it matters
Our team is five or six people at most.	Yes, team of 5 members	Beyond six, split into two teams.
We counted our communication paths: $n(n - 1) \div 2 =$ _____	Yes, 10 paths	Five people share 10 paths; ten share 45.
We added explicit work for integrating the pieces of the slice.	No, see the integration work explanation below the table.	Breaking work into pieces hides the cost of putting it back together.
We did not assume that more people means proportionally more output.	Yes	Coordination grows faster than headcount.
We estimated for the team we have.	Yes	Not for a larger team we wish we had.
We adjusted, not copied, timelines from much larger teams.	Not applicable. Our comparable was one person, smaller than our team, so we scaled its effort up (3D) rather than copying a larger team's timeline down.	Their effort includes their coordination costs.

Integration work explanation (Step 4, row 3).

The answer stays No until the work below is priced. Coordination and technical integration are different things, and only the first is covered so far. The 25 to 35 percent uplift in the analogous estimate covers coordination among five people, and the 6 to 12 hour function point rate is an all-in rate from design through testing, but neither shows that the integration work was counted.

Connecting the outside event source and defining the import: assumed inside the 1 external interface file and the top of the function point rate.

Merging feeds and resolving straightforward duplicates, meaning the same event arriving twice in a recognisable form: assumed inside the external output already counted for the unified calendar.

Standardizing dates, times, locations and categories across sources, including the two cities: assumed inside the function point rate.

Stale, conflicting and unrecognisable duplicate listings, and recovery when a source fails: NOT YET PRICED. This is the one line with no home in the current numbers, and it cannot be priced until we fix how many sources the first release reads.

Wiring the calendar, detail page, ranked feed and saved events together: assumed inside the function point rate.

End-to-end tests, data-failure handling and deployment: assumed inside the function point rate.

No separate hours have been added for any line, so nothing is counted twice. Because one line remains unpriced, the Step 6 range, expected hours, cost and schedule are provisional and must be revisited once the source boundary is fixed.

Step 5. Compare your two estimates (Part 2, Slides 31 and 32)

Two estimates almost never match. The gap is useful: it points to the assumption you are least sure about. Your job is to find that assumption and name it.

	Method 1	Method 2
Method	Function points	Analogous Estimation
Estimate in hours (low to high)	498 hours to 996 hours	394 hours to 595 hours
Middle value in hours	747 hours (83 FP x 9h)	491 hours
Input that drives it most	Productivity rate (hours per fp)	Hours per person-month assumed for the comparable (70), then the fraction of the release matched
Is that input measured or assumed?	Assumed	Assumed

Gap: larger middle value $\div$ smaller middle value = $747 \div 491 = 1.52$ (an absolute difference of 256 hours)

If one estimate is more than about twice the other, recheck your counts and conversions before explaining the gap.

Questions to answer

1. Do your two methods depend on the same assumption? If so, their agreement proves little.

No, the function points method starts from our internal design counts (inputs, reports, results, etc.) in step 1. multiplied by productivity rates. The Analogous method starts from external market historical data scaled to our scope and adjusted for 5-person team coordination costs.

2. Which single input moves the answer most? Try changing it and see.

The productivity rate in function points moves the estimate by 498 hours, from 498 at 6 h/FP to 996 at 12. In the analogous method, holding the coordination uplift fixed at 30 percent and moving the matched-release fraction from 50 to 70 percent takes the estimate from 409.5 to 573.3 hours, a difference of 163.8 hours. The full 394 to 595 range varies the uplift as well, between 1.25 and 1.35, so that wider 201.6 hour spread cannot be attributed to the fraction alone. The analogous method's largest lever is the 70 hours per person-month assumed for Baio's side project: at 100 hours per person-month the range would be 563 to 851 hours, and at a full-time 160, 900 to 1,361 hours, more than the fraction or the uplift can move it.

3. What information would narrow the gap, and when could you get it?

Our first one or two actual software-development sprints would give early effort observations, but they would not establish a reliable rate. We need hours recorded against completed stories, and Slide 26 sets at least three completed sprints by the same team before velocity means anything. Course-portal documentation sprints do not count as GPI development velocity. More data should narrow the 6 to 12 hour range; it will not eliminate the uncertainty.

Our explanation of the gap (one paragraph)
Function point yielded a middle value of 747 hours, whereas the Analogous method yielded a middle value of 491 hours, creating a gap of 1.52x (under 2.0x). The primary driver of the gap is that our Function point estimate assumes a wide productivity rate of 6 to 12 hours per point due to lack of sprint data. Meanwhile, the Analogous method relies on our assumed Fraction of their release our slice matches and a 70-hour per person month. Both methods start from completely independent information, so the gap is not a slip in the arithmetic; it is the distance between two assumptions we have never tested. Story points, which we completed as a cross-check, gave 463 to 925 hours for the current 37-point slice and sits between the two, but its velocity is an assumption as well, so it settles nothing. What would narrow the gap is real hours logged against finished stories, and Slide 26 sets three completed sprints as the minimum before that data means anything. On sensitivity: holding the coordination uplift at 30 percent and moving the feature match from 50 to 70 percent takes the analogous estimate from 409.5 to 573.3 hours, a swing of 163.8 hours. The wider 394 to 595 spread varies the uplift as well, so the fraction alone does not account for all of it.

Example: function points gave a middle value of 472 hours and the analogous method gave 576 hours, a gap of about 1.2. The two are fairly close, and they start from different information, so the agreement is meaningful. The largest uncertainty is the productivity rate, which is our assumption, and the fraction of ParkEasy's release that our slice represents.

Step 6. Present the estimate (Part 2, Slide 33)

This is the table your sponsor reads. It gives a range instead of a single number, shows what the range depends on, and is dated because estimates change as you learn more. It becomes the Estimation Appendix on your Sprint 2 portal page.

How to fill it in

1. Range: the lowest and highest hours across both methods.
2. Expected value: set O to your low end and P to your high end. Choose M, your most likely value, somewhere between them and explain why. Then calculate $(O + 4M + P) \div 6$.
3. Cost: multiply hours by a loaded hourly rate. State the rate and where it came from, such as published wage data plus an overhead percentage you choose.
4. Calendar time: divide hours by (team size $\times$ productive hours per person per week). State the productive hours you assume.

Field	Our estimate
Slice estimated	User/Event goer features
Range in hours (low to high)	394 hours to 996 hours
Expected hours: $(O + 4M + P) \div 6$, and why we chose M	$(394 + 4(657) + 996)/6 = \mathbf{669.67 \text{ expected hours}}$ For M we took the midpoint of the lowest and highest ends of the range, which gave a first value $M1 = 695$, then the midpoint of the two methods' middle values, which gave $M2 = 619$. We settled on the midpoint of those two, $M = 657$. $M = 657$ is a planning judgment, not a measured value. We think an outcome near 657 hours is plausible because the function point inventory is complete and countable while its productivity rate is not ours, and the analogous figure rests on a comparable of the right shape but three assumed multipliers. 657 sits where neither method's weakness dominates. If the duplicate and conflict handling in Step 4 is priced separately, M should move up.
Cost range and loaded rate used	Hourly rate: 51.36, according to ZipRecruiter, is the average salary for Software Engineers in Texas. Overhead percentage: 20% Loaded Hourly Rate: 61.63 Source: https://www.ziprecruiter.com/Salaries/It-Software-Engineer-Salary--in-Texas#TableView • Low: $394 \times 61.63 \approx \\$24,282$ • Expected: $669.67 \times 61.63 \approx \\$41,272$ • High: $996 \times 61.63 \approx \\$61,383$

Field	Our estimate
Calendar estimate in weeks, and hours per week assumed	Expected: 13.39 ~ 14 weeks Assumed 10 productive hours per person per week. Low end: $394 \div 50 \approx 7.9$ weeks High end: $996 \div 50 \approx 19.9$ weeks. This assumes all five people are available continuously at 10 productive hours a week. It is a production estimate for the slice, not a commitment that this team can deliver it inside the remaining Fall 2026 term. Part 1's 6 to 12 months is a rough comparator for a production-ready full first release; the 13.4 weeks here is the event-goer slice alone at the stated capacity, and neither is a Fall 2026 delivery promise. The Sprint 2 business case should reconcile its pilot scope and milestones against this rather than read it as a delivery date.
Methods used	Analogous and Function Points. Story points (3C) was completed as a cross-check, not as a declared method.
Key assumptions	• Productivity rate: 6 to 12 hours per function point, taken from Part 2, Slide 15. We have no sprint history, so we haven't measured this. • Scope: the scope stays as listed in Step 1. • 70 hours per person-month for Baio's side-project effort on Upcoming, rather than a full-time 160. • Our slice is 50 to 70 percent of Upcoming's first release. • A 25 to 35 percent coordination uplift for five people. • 20 percent overhead on the ZipRecruiter hourly figure. • 10 productive hours per person per week, which drives every week figure. • Velocities of 4, 6 and 8 points per sprint in the story point cross-check. • Event supply: one outside source feeding the unified calendar, in addition to organizer posting. The number of sources Release 1 reads, if any, is not yet fixed. • The seven story point values were drafted for this activity and have not been sized in a planning poker session by the whole team.
Team size assumed	5 team members
What would change this estimate	• Adding more productive hours per person. • Pricing the stale-listing, conflict and source-failure handling left unpriced in Step 4, which would move M up. • The event-supply decision. If Release 1 relies on organizer posting and team curation only, the external interface file and the merged-calendar output drop out: about 75 function points, 450 to 900 hours. Each additional distinct outside source adds 7 function points, about 42 to 84 hours. • Real productivity and velocity data after three completed sprints, which Slide 26 sets as the minimum • Scope changes • Team changes
Date of this estimate	29 September 2026

Example, “Buy a semester permit” slice: range 354 to 708 hours. With $M = 520$, expected hours = $(354 + 2,080 + 708) \div 6 \approx 524$. At an assumed loaded rate of \$60 per hour, cost is about \$21,000 to \$42,500. With 5 people at an assumed 30 productive hours a week (150 team hours), the expected 524 hours is about 3.5 weeks.

Going further (optional)
Once your slice estimate is solid, you can use it to make a rough estimate of the whole project. If your slice holds about one fifth of your backlog points, the whole project is roughly five times the slice. Treat that number as a first rough guess, and repeat this activity on more slices as you learn more.

Before you submit

1. Your slice is named, with its stories, screens, data, and boundaries written down.
2. Both estimates are for the same slice and in hours, with the conversions shown.
3. Every assumed number is labeled as an assumption.
4. Every sourced number has a source you can link.
5. The team-size check is complete, and any No has a fix or explanation.
6. The final estimate is a dated range, not a single number.
7. The individual estimation memo is your own work, written without AI.